

Disciplina de



Professora Lisiane Reips

Agenda de hoje



- React com Upload de Imagens
 - Conceitos Fundamentais & File API
 - Estratégias de Preview de Imagem
 - Tipos de arquivos suportados



React com Upload de Imagens: Conceitos Fundamentais & File API

O processo de upload de arquivos no ecossistema Web moderno é estruturado em quatro etapas principais, conectando a interface do usuário ao armazenamento persistente:

1. Seleção do Arquivo: o usuário escolhe o arquivo através de um elemento `<input type="file">` ou por mecanismos de arrastar e soltar (drag-and-drop).
2. Leitura em Memória: o JavaScript acessa o arquivo de forma assíncrona, podendo convertê-lo em Base64 ou gerar uma URL em memória.

Obs.: a Base64 é um método de conversão de dados binários (como imagens, documentos ou arquivos executáveis) em texto simples, que utiliza um conjunto de 64 caracteres seguros (letras maiúsculas e minúsculas, números de 0 a 9, e símbolos + e /) para representar qualquer informação de forma legível por sistemas que lidam apenas com texto.

3. Empacotamento e Envio: os dados binários são organizados via objeto `FormData` e transmitidos ao servidor através de requisições multipart.

4. Processamento e Destino: o ecossistema backend valida a integridade do arquivo e o encaminha para o armazenamento final (disco local, banco de dados ou Blob Storage).

Atenção com a Segurança do Navegador

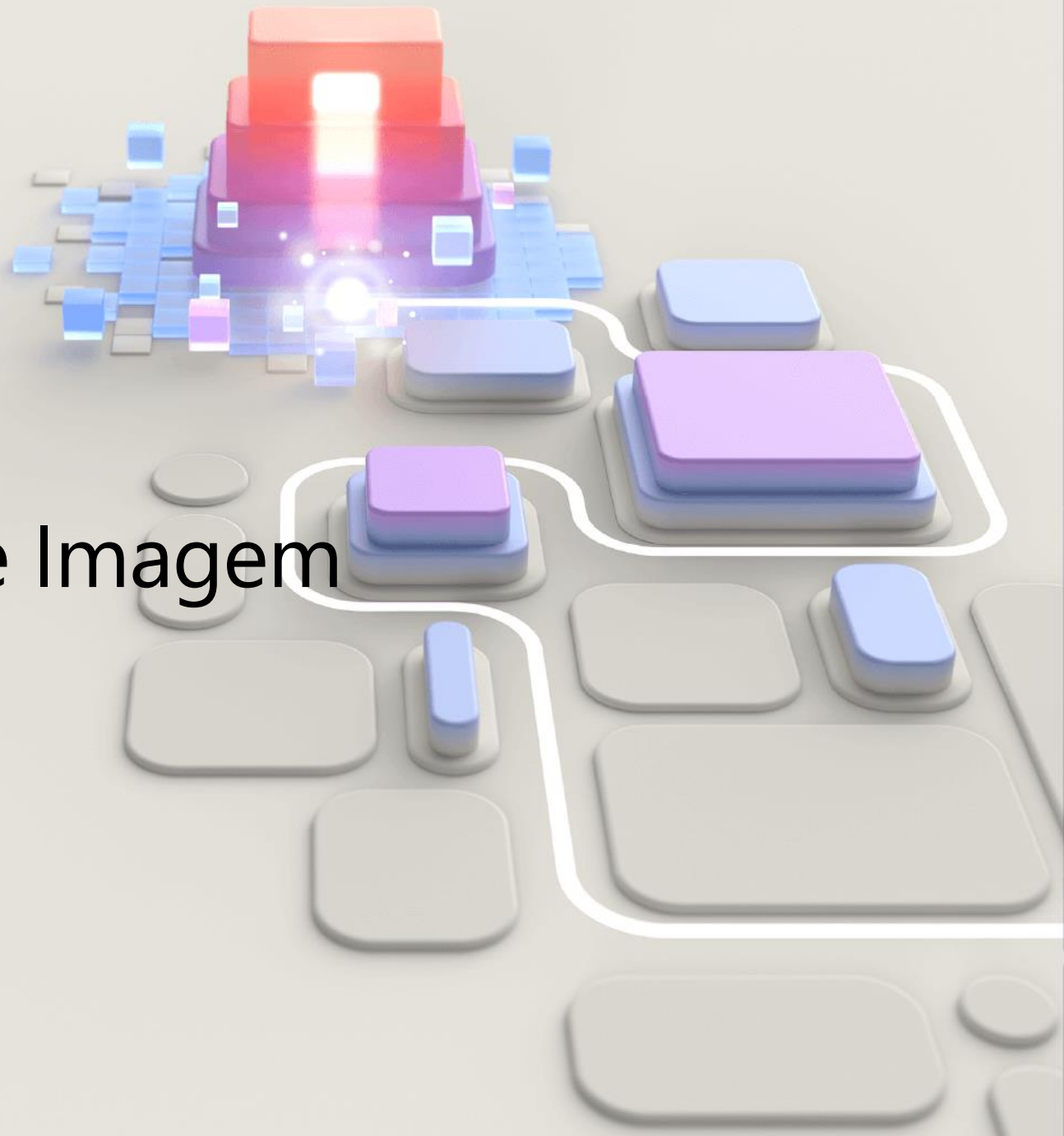
Por questões rígidas de segurança e sandbox, o JavaScript não possui autorização para ler arquivos diretamente do sistema de arquivos local sem a ação explícita e consciente do usuário. O acesso é restrito unicamente aos arquivos selecionados via input ou drag-and-drop.

No navegador, o objeto global `File` expõe metadados para validações prévias no lado do cliente:

- > `file.name`: O nome original do arquivo (ex: foto.jpg).
- > `file.size`: O tamanho do arquivo mensurado estritamente em bytes.
- > `file.type`: O tipo MIME do arquivo (ex: image/jpeg, image/png).
- > `file.lastModified`: O timestamp que indica a última modificação do arquivo.

O **tipo MIME** (do inglês Multipurpose Internet Mail Extensions) é um padrão que identifica a natureza e o formato de um arquivo transmitido pela Internet. Ele orienta navegadores, servidores e sistemas operacionais sobre como processar, abrir ou exibir o arquivo corretamente.

Estratégias de Preview de Imagem



Fornecer um feedback visual imediato antes do envio melhora a experiência do usuário. O ecossistema frontend dispõe de dois métodos principais para essa finalidade:

Método 1: `URL.createObjectURL()` gera uma string contendo uma URL temporária vinculada diretamente à sessão atual da memória do navegador. É performático porque não realiza conversões pesadas de strings.

Método 2: FileReader (Conversão para Base64) efetua a leitura assíncrona do arquivo e gera uma representação em texto sob o formato Data URL (Base64). É um método um pouco mais lento para arquivos volumosos, porém, é indispensável quando há a necessidade de trafegar a imagem serializada dentro de um corpo JSON comum ou salvá-la diretamente de forma textual.

Tipos de arquivos suportados



Os sistemas podem aceitar diferentes tipos de arquivos, como:

imagens (.png, .jpg)

documentos (.pdf, .docx)

vídeos (.mp4)

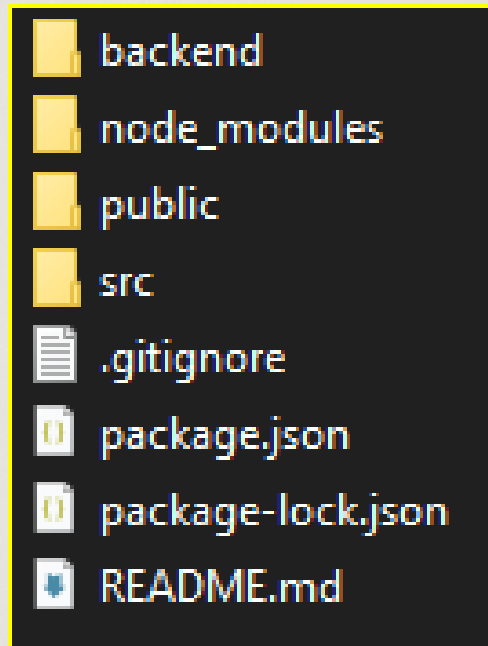
áudios (.mp3)

A validação dos formatos é importante para garantir segurança e compatibilidade da aplicação.

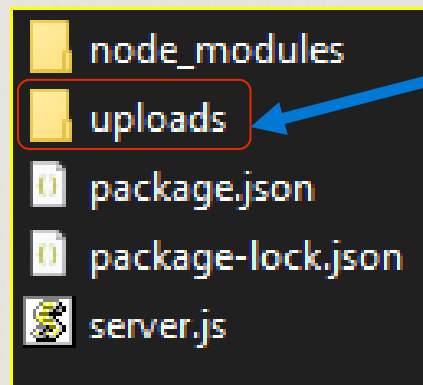
Vamos ver isso tudo na prática?



Vamos reorganizar nossos diretórios, para uma boa prática de reutilização de códigos. Nosso **meuapp** ficou assim:

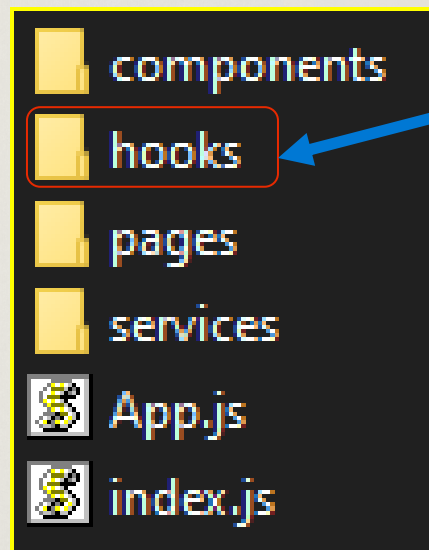


O nosso diretório `meuapp/backend` ficou assim:



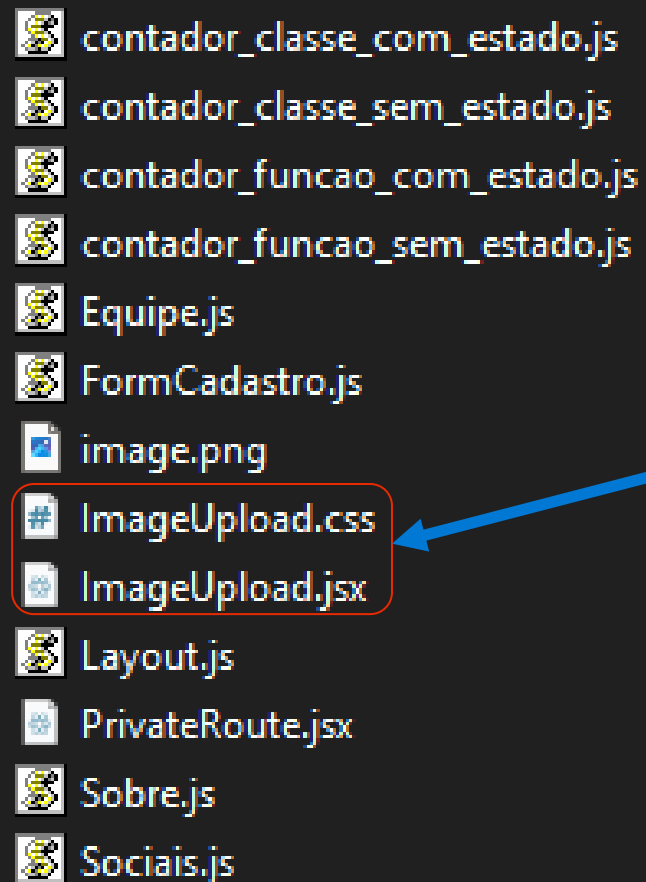
Criamos essa pasta para mantermos nossas imagens de upload ali

O nosso diretório `meuapp/src` ficou assim:



Criamos essa pasta para mantermos separados nossos hooks

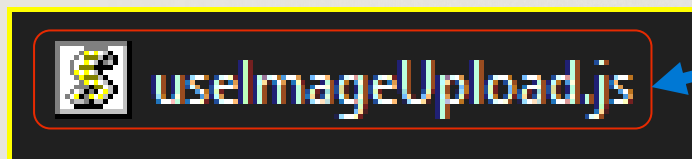
O nosso diretório `meuapp/src/componentes` ficou assim:



-  contador_classe_com_estado.js
-  contador_classe_sem_estado.js
-  contador_funcao_com_estado.js
-  contador_funcao_sem_estado.js
-  Equipe.js
-  FormCadastro.js
-  image.png
-  ImageUpload.css
-  ImageUpload.jsx
-  Layout.js
-  PrivateRoute.jsx
-  Sobre.js
-  Sociais.js

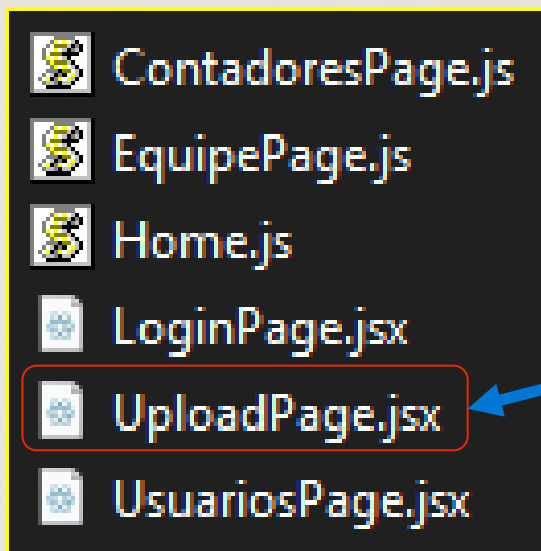
E aqui nós vamos trabalhar
nosso componente de
upload de imagens

O nosso diretório `meuapp/src/hooks` ficou assim:



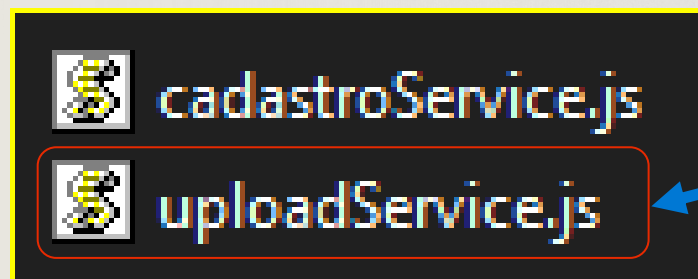
Criamos nosso Hook customizado. Vejam que todos iniciam com `use`. o React usa esse prefixo para identificar e aplicar as regras dos hooks

O nosso diretório `meuapp/src/pages` ficou assim:



Criamos uma página para mostrar o nosso upload.

O nosso diretório `meuapp/src/services` ficou assim:



Criamos nosso servisse do frontend, que vai rodar no navegador e fazer a requisição HTTP para o backend

Agora que sabemos o que mudou em nosso projeto, vamos entender a parte lógica dos códigos. Vamos começar pela instalação do Multer.

```
npm install multer
```

software que se encontra
entre o sistema operacional e
os aplicativos nele executados

O Multer é um middleware do Node.js/Express especializado em receber arquivos enviados via multipart/form-data.

Agora vamos adicionar as dependências no nosso backend/server.js.



```
JS server.js  X
backend > JS server.js > ...
1  const express = require("express");
2  const cors = require("cors");
3  const jwt = require("jsonwebtoken");
4  const multer = require("multer");
5  const path = require("path");
```


E incluir, ainda no nosso backend/server.js, as configurações do Multer.

```
6
7  const app = express();
8  app.use(cors());
9  app.use(express.json());
10
11  const SECRET = "segredo_jwt";
12
13  // — Configuração do Multer —
14
15  // diskStorage salva o arquivo no disco com nome único.
16  // Alternativa: memoryStorage() para guardar em buffer e enviar a um serviço de nuvem.
17  const storage = multer.diskStorage({
18    destination: (req, file, cb) => {
19      cb(null, "uploads/"); // pasta local – crie-a ou o Multer lança erro
20    },
21    filename: (req, file, cb) => {
22      // Ex.: "1716041234567-imagem.png"
23      const uniqueName = `${Date.now()}-${file.originalname}`;
24      cb(null, uniqueName);
25    },
26  });
27
```

Aqui é o timestamp (em milissegundos) vai garantir que dois arquivos com o mesmo nome original nunca se sobrescrevam

E configurar o filtro e os limites do Multer.

```
28 // Filtro: rejeita arquivos que não sejam imagem no próprio backend
29 function fileFilter(req, file, cb) {
30   const allowed = ["image/jpeg", "image/png", "image/gif", "image/webp"];
31   if (allowed.includes(file.mimetype)) {
32     cb(null, true);
33   } else {
34     cb(new Error("Formato de imagem inválido."), false);
35   }
36 }
37
38 const upload = multer({
39   storage,
40   fileFilter,
41   limits: { fileSize: 5 * 1024 * 1024 }, // 5 MB
42 });
```

Entendendo o fluxo do filtro:

arquivo chega

↓
fileFilter → é imagem? → não → rejeita com erro

↓ sim
limits → é menor que 5MB? → não → rejeita com erro

↓ sim
storage → salva no disco com o nome definido

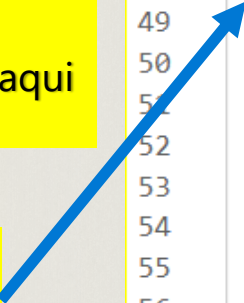
E configurar o filtro e os limites do Multer.

Todo middleware do Express recebe esses 3 parâmetros:
req: a requisição que chegou
res: a resposta que será enviada
next: uma função que diz "pode continuar para a próxima etapa"

Obs.: se o `next()` não for chamado, a requisição trava aqui e nunca chega à rota

O token JWT viaja no cabeçalho HTTP chamado `Authorization`. Se não vier nenhum cabeçalho, já bloqueia na hora com status `401` (não autorizado).

```
43
44 // Middleware de autenticação JWT (reutilizado)
45
46 function authMiddleware(req, res, next) {
47   const authHeader = req.headers.authorization;
48   if (!authHeader) return res.status(401).json({ erro: "Token ausente." });
49
50   const token = authHeader.split(" ")[1]; // "Bearer <token>"
51   try {
52     req.usuario = jwt.verify(token, SECRET);
53     next();
54   } catch {
55     res.status(401).json({ erro: "Token inválido." });
56   }
57 }
58
```



Na sequência, o cabeçalho chega como `Bearer eyJhbGci....`. `split(" ")` e divide pelo espaço → array com 2 posições: `[0] = "Bearer"` · `[1] = o token`. O `[1]` descarta o "Bearer" e pega só o JWT.

Então, `jwt.verify` faz duas verificações ao mesmo tempo:

- ✓ token foi assinado com o SECRET correto?
- ✓ token ainda não expirou?

Se válido → salva em `req.usuario` e chama `next()`

Agora, se o token for falso, adulterado ou expirado, o `jwt.verify` lança uma exceção que cai no `catch` e retorna status `401` para o cliente

Continuando no backend/server.js

```
59 ∨ // Rota de upload de imagem, protegida por JWT e processada pelo Multer.
60
61 // "uploads/" servido como arquivos estáticos para o frontend acessar a URL
62 app.use("/uploads", express.static(path.join(__dirname, "uploads")));
63
64 // upload.single("imagem"): processa o campo "imagem" do FormData
65 ∨ app.post(
66   "/upload",
67   authMiddleware, // exige JWT válido
68   upload.single("imagem"), // nome deve bater com formData.append("imagem", file)
69   ∨ (req, res) => {
70     ∨ if (!req.file) {
71       | return res.status(400).json({ erro: "Nenhum arquivo recebido." });
72     }
73
74     // Monta URL pública para o frontend exibir a imagem
75     const url = `http://localhost:3001/uploads/${req.file.filename}`;
76     res.json({ url });
77   }
78 );
79
```

Continuando no backend/server.js

```
80 // Tratamento de erros do Multer (tamanho, tipo)
81 app.use((err, req, res, _next) => {
82   if (err instanceof multer.MulterError || err.message) {
83     return res.status(400).json({ erro: err.message });
84   }
85   res.status(500).json({ erro: "Erro interno." });
86 });
87
88 // Rota de login (igual à original)
89
90 const usuarioFake = { email: "admin@email.com", senha: "123456" };
91
92 app.post("/login", (req, res) => {
93   const { email, senha } = req.body;
94   if (email !== usuarioFake.email || senha !== usuarioFake.senha) {
95     return res.status(401).json({ erro: "Usuário inválido" });
96   }
97   const token = jwt.sign({ email }, SECRET, { expiresIn: "1h" });
98   res.json({ token });
99 });
100
101 app.listen(3001, () => console.log("Servidor rodando na porta 3001"));
102
```


Agora, veremos o restante do desenvolvimento do projeto em nosso código e, na sequência, o código estará disponível no repositório do GitHub e pode ser clonado ou atualizado via git pull.

